Enterprise Application Management Governance & Operating Model (Microsoft Endpoint / Intune)

Implementation Solution Guide

Version 1.0.0 · 2026-07-20

Contents

Executive Summary 5

The problem . . . 5

The solution . . . 5

What you get . . . 5

Business impact . . . 5

Bottom line . . . 6

Solution Overview 6

The problem this solves . . . 6

What this solution is NOT . . . 6

The seven components of the operating model . . . 7

How governance connects to Intune and the security stack . . . 8

The standard deployment model . . . 9

Limitations and honest scope boundaries . . . 10

A note for Government tenants . . . 10

Requirements 10

R1. A named executive sponsor with authority over application decisions . . . 11

R2. Intune Administrator role (for the initial build) . . . 11

R3. Least-privilege operating roles (the steady-state model) . . . 11

R4. Ability to create Microsoft Entra security groups . . . 12

R5. Microsoft Graph PowerShell SDK and the required delegated scopes . . . 12

R6. A system of record for the Application Register . . . 13

R7. An existing change management process (or a decision that none exists) . . . 14

R8. An application discovery data source beyond Intune . . . 14

R9. Nominated individuals for the four RACI roles . . . 15

R10. Licence and entitlement data access . . . 15

Global Requirements — Intune Solutions (category-level) 15

G1. Microsoft Entra ID tenant . . . 15

G2. Microsoft Intune subscription . . . 16

G3. Administrative roles . . . 16

G4. Device enrollment in Intune . . . 16

G5. Pilot security group . . . 16

G6. Supported Windows baseline . . . 16

G7. Network access to Microsoft cloud endpoints . . . 16

Implementation 17

Step 1 — Baseline the current application estate . . . 17

Step 2 — Create the Application Register . . . 18
Step 3 — Define the four accountabilities (the RACI) . . . 20
Step 4 — Create the four role security groups in Entra ID . . . 21
Step 5 — Create the scope tags and the least-privilege Intune role . . . 22
Step 6 — Create the deployment ring groups . . . 24
Step 7 — Publish the Application Standards Document . . . 25
Step 8 — Stand up the intake workflow . . . 26
Step 9 — Stand up the Application Review Board . . . 27
Step 10 — Publish the change classification matrix and the CAB interface . . . 28
Step 11 — Define the support model and the runbook standard . . . 29
Step 12 — Stamp governance metadata onto the existing estate . . . 31
Step 13 — Pilot the model for two full board cycles . . . 31
Step 14 — Expand to the whole estate and enter steady state . . . 32
Testing & Validation 33
Gate 1 — The structural objects exist (automated) . . . 33
Gate 2 — The custom role grants exactly the intended rights (manual) . . . 33
Gate 3 — Broad grants have been removed (manual) . . . 34
Gate 4 — Every application is governed (automated) . . . 34
Gate 5 — The intake gate returns non-conforming requests (behavioural) . . . 34
Gate 6 — The board's decision rights are real (behavioural) . . . 34
Gate 7 — Ringed deployment and rollback work (behavioural) . . . 35
Gate 8 — Change classification is applied consistently (behavioural) . . . 35
The overall pass condition . . . 35
Troubleshooting 36
Technical — scripts and objects . . . 36
Organisational — the process . . . 37
Best Practices 38
Secure the sponsor before the process, not after . . . 38
Make the front door the only door . . . 38
Pre-approve the routine so the board can judge the exceptional . . . 39
Name individuals, never teams . . . 39
Treat the Security Reviewer as non-negotiable for classified data . . . 39
Ring everything; assign nothing to All Users . . . 39
Make rollback approval-free and cheap . . . 39
Reconcile against reality, not against your own deployment records . . . 39
Publish the scorecard where decisions are made . . . 40

Let the standards document be a living test, not a monument 40
Retire deliberately; a smaller estate is the goal 40
Deliverables Summary 40
Documents 40
PowerShell scripts 41
Artefacts the model produces (and you keep) 42
How the pieces fit 42
Appendix A — PowerShell Scripts 43

Executive Summary

The problem

Most organisations can package and deploy an application. Very few can say **who owns it, who approved it, on what evidence, and who supports it.** The result is an application estate that quietly accumulates risk: applications assigned to groups nobody maintains, applications never security-reviewed, applications deployed beyond their licence entitlement, and applications supported by a single engineer who "just knows how it works." Better packaging tools do not fix this, because the missing capability is not technical — it is an **operating model**: defined roles, a controlled intake path, a decision forum with real authority, published standards, risk-tiered change control, and a support model with genuine escalation.

The solution

This solution is that operating model, built for Microsoft Intune and Entra ID and deliberately scoped to the **governance layer** — not packaging mechanics. It defines seven components: application ownership and RACI; an intake and request workflow; an Application Review Board with explicit decision rights; published standards for what "approved" means; risk-tiered change management with a Change Advisory Board interface; a tiered operational support model; and the metrics that prove the model works. Critically, each governance decision is expressed as a **concrete object inside the platform** — scope tags, a least-privilege custom role, named security groups, deployment rings, and audit evidence — so the governance is enforced by Intune rather than living only in a document.

What you get

- A complete, click-by-click **implementation guide** that stands up every role, group, scope tag, register, forum, standard, and change process, in order, with every object explicitly named.
- Ready-to-use **artefact definitions**: the Application Register schema, the RACI matrix, the standards document structure, the change classification matrix, the support matrix, the runbook template, and the ARB decision record.
- **Five PowerShell scripts** — read-only estate discovery, model validation, a monthly governance scorecard, an approval-evidence exporter for audit, and a gated (`-WhatIf`) remediation script that stamps ownership and scope tags onto the existing estate.
- A pilot-first method (**Assess** → **Configure** → **Pilot** → **Validate** → **Expand**) with eight explicit pass/fail gates, so you prove the model on one business unit before committing the whole estate.

Business impact

- **Accountability, everywhere:** every application has a named business owner, technical owner, security reviewer, and deployment engineer — leavers no longer orphan applications silently.
- **Defensible decisions:** every production application carries an auditable trail of who approved it and on what evidence, satisfying security and compliance review.

• **Lower risk deployments:** ringed rollout and approval-free rollback turn a broken package into a contained Ring 1 incident instead of a fleet-wide outage.

• **Least-privilege operations:** application change rights are separated from device, compliance, and Conditional Access rights via a scoped custom role — a measurable reduction in standing privilege.

• **A smaller, cheaper estate:** deliberate retirement of unowned applications reduces licence cost, security surface, and support load. The delta from baseline is the business case.

Bottom line

This turns "who owns this app, and who said we could deploy it?" from a question nobody can answer into a governed, measured, and auditable operating model — built on the Intune and Entra platform you already run, at a fraction of the cost of a consulting engagement, and reusable as a permanent capability your team keeps.

Solution Overview

The problem this solves

Most organisations can *package* an application. Very few can answer these questions about the applications already in their estate:

• **Who owns this app?** Not "who installed it" — who is accountable when it breaks, when the vendor ships a breaking change, or when Security asks whether it still needs to be there.

• **Who approved it, and on what evidence?** Was there a security review? A licence check? Is there a record, or did somebody just package it because a director asked?

• **What happens when it changes?** Who decides the app can be pushed to 12,000 devices? Who can stop the rollout? Who signs off the rollback?

• **Who supports it?** When the service desk gets a ticket, does it go to L1, L2, or the vendor — and is there a runbook, or is there one engineer who "just knows"?

When these questions have no answer, the estate accumulates **orphaned applications** (assigned to groups nobody maintains, owned by people who left), **unreviewed applications** (no security or licensing evidence), and **unsupportable applications** (no runbook, one hero engineer). Packaging quality does not fix this. Better tooling does not fix this. What fixes it is an **operating model**: named roles, a defined intake path, a decision forum with authority, published standards, a change process with risk tiers, and a support model with real escalation paths.

This solution is that operating model. It is deliberately a *governance* product, not a technical one.

What this solution is NOT

This package does **not** teach application packaging or deployment mechanics. It assumes you either already have those capabilities or will acquire them separately. Specifically, it does **not** cover:

- How to build a Win32 `.intunewin` package, write detection rules, or author requirement rules.
- How to configure third-party patch automation.
- How to troubleshoot a failing app install (exit codes, Management Extension logs, ESP blocks).
- How to configure WDAC policies, Defender for Endpoint, or App Protection policies in detail.

What it **does** cover is the layer above all of that: **who decides, on what basis, with what evidence, and who is accountable afterwards** — plus exactly how those decisions are expressed as objects inside Microsoft Intune so the governance is enforced rather than aspirational.

The seven components of the operating model

The model has seven components. Each is defined in this package, and each produces a **named, durable artefact** the organisation keeps.

1. Ownership and RACI

Every application in the estate has **four named accountabilities**:

Role	Accountable for	Typically held by
Application Owner	The business need. Whether the app should exist at all, budget, licence count, renewal, and retirement.	A business function lead (Finance, HR, Clinical, Legal).
Technical Owner	The package, its configuration, its versions, its runbook, and its supportability.	An endpoint engineer or platform team member.
Security Reviewer	The risk assessment: data handled, network behaviour, vendor posture, control coverage.	Information Security.
Deployment Engineer	Executing the approved change: rings, timing, rollback, evidence.	Endpoint operations.

The artefact is the **Application Register** — the authoritative list of every application, its four owners, its review status, and its lifecycle state. Section 3 (Step 2) builds it, and `APPOPS-Discovery.ps1` seeds it from your live tenant.

2. Intake and request workflow

A defined front door: how a new application enters the estate. Every request carries a **business justification**, a **data classification**, a **licence/entitlement position**, and a **named Application Owner** before it is allowed to consume engineering time. Requests that cannot supply those four things are returned, not queued.

The artefact is the **Application Intake Request Form** and the **intake queue** that feeds the review board.

3. The Application Review Board (ARB)

A standing decision forum with **explicit decision rights**. The ARB is not a discussion group — it is the body that can say *approved*, *approved with conditions*, *deferred pending evidence*, or *rejected*, and whose decision is binding. It meets on a fixed cadence, has defined membership and quorum, and requires a defined **evidence pack** before an item is tabled.

The artefact is the **ARB Decision Record** — one entry per decision, retained for audit.

4. Standards and policy

A published definition of what **"approved"** means, so that approval is a test rather than an opinion. Covers the security review criteria, the licensing and entitlement check, the packaging and naming standards the estate must conform to, and the supportability bar an app must clear before it is allowed into production.

The artefact is the **Application Standards Document** and the **naming convention** applied inside Intune.

5. Change management

Applications change constantly — new versions, new configuration, new assignments, retirement. The model classifies every change into a **risk tier**, and each tier carries a defined approval path, deployment ring policy, change window, and rollback authority. It also defines the interface to a formal **Change Advisory Board (CAB)** where one exists, so application changes are represented in enterprise change control without every minor version consuming a CAB slot.

The artefact is the **Application Change Classification Matrix** and the **deployment ring definitions** expressed as Entra groups in your tenant.

6. Operational support model

Who answers the ticket. A defined **L1 / L2 / L3 split** with explicit boundaries, escalation triggers and paths, response and resolution targets per severity, and — critically — **named ownership of every runbook**, so support knowledge is an organisational asset rather than a personal one.

The artefact is the **Application Support Matrix** and one **Application Runbook** per production application.

7. Metrics and evidence

The measures that prove the model is working, and the evidence trail that proves it to an auditor. Coverage metrics (what fraction of the estate is owned, reviewed, ringed), flow metrics (intake to decision time), quality metrics (change success rate, rollback rate), and risk metrics (orphaned apps, overdue reviews).

The artefact is the **Application Governance Scorecard**, produced by `APPOPS-Report.ps1`.

How governance connects to Intune and the security stack

Governance that lives only in a document is decoration. This model expresses each governance decision as a **concrete object inside Microsoft Intune and Microsoft Entra ID**, so the platform enforces it:

Governance decision	Expressed in the platform as
"This app has a technical owner."	The owner's name and the ARB decision reference recorded in the app's **Notes** field in Intune, stamped by `APPOPS-Remediation.ps1`.
"This app is approved for production."	Application of the Intune **scope tag** `Scope – App Management – Prod`. Unapproved apps carry `Scope – App Management – Pilot` only.
"Only the deployment engineers may change apps."	A custom Intune role, `Role – App Deployment Engineer`, scoped by scope tag, replacing broad Intune Administrator grants.
"Every rollout is ringed."	Three assigned Entra security groups — `PILOT – App Ring 1 – Validation`, `PILOT – App Ring 2 – Early Adopters`, `PROD – App Ring 3 – Broad` — and a rule that no app may be assigned directly to *All Users* or *All Devices*.
"The change was authorised and by whom."	Intune **audit logs**, exported into the ARB evidence pack by `APPOPS-Export-ApprovalEvidence.ps1`.

The model also defines the **interface** to your security controls without re-specifying them:

• **Application Control (WDAC / App Control for Business)** — the ARB's approval decision is the *input* to your allow-list. The model defines who submits an app for allow-listing, when in the lifecycle it happens (after security review, before Ring 2), and who is accountable for the resulting policy change. It does not tell you how to author WDAC policy.

• **Microsoft Defender for Endpoint** — MDE's software inventory is a **discovery input** to the Application Register: apps present on endpoints but absent from the Register are, by definition, ungoverned. The model defines the reconciliation cadence and who acts on the exceptions.

• **App Protection Policies (MAM)** — the intake form's data-classification answer determines whether an application requires an App Protection Policy before approval. The model defines the trigger and the accountable reviewer, not the policy settings.

The standard deployment model

This solution follows the same shape as every other solution in the category: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

• **Assess** — inventory the existing estate, identify orphaned and unowned applications (`APPOPS-Discovery.ps1`).

• **Configure** — stand up the roles, groups, scope tags, register, standards, and forum.

• **Pilot** — run the model over a limited scope (one business unit or one application family) for two full ARB cycles.

- **Validate** — measure against the governance gates in Section 5 (`APPOPS-Validation.ps1`).
- **Expand** — extend to the whole estate and move to steady-state operation with the scorecard.

Limitations and honest scope boundaries

- **This is an operating model, not a purchase of maturity.** It works only if the organisation actually assigns the named roles and holds the forum. A model nobody attends is a document.
- **It requires an accountable executive sponsor.** Decision rights are only real if somebody senior will back the ARB when a director's favourite application is rejected. If you cannot identify that sponsor, stop and secure it before Step 1.
- **It does not replace enterprise change management** where one exists. It defines the *interface* to CAB, so application changes are handled proportionately rather than duplicated.
- **Metrics require the register to be maintained.** The scripts can measure what is in Intune; they cannot measure business ownership that was never recorded.

A note for Government tenants

The process, roles, decision rights, and artefacts in this solution are **identical** for Commercial and US Government (GCC / GCC High / DoD) tenants. The only difference is the **portal URLs** you navigate to:

Purpose	Commercial	US Government
Intune admin centre	`https://intune.microsoft.com`	`https://endpoint.microsoft.us`
Entra / Azure portal	`https://entra.microsoft.com` / `https://portal.azure.com`	`https://portal.azure.us`
Sign-in authority	`https://login.microsoftonline.com`	`https://login.microsoftonline.us`
Microsoft Graph endpoint	`https://graph.microsoft.com`	`https://graph.microsoft.us`

Every script in this package accepts a `-GraphEnvironment` parameter (`Global` or `USGov`) so it targets the correct sovereign endpoint. Where a specific Intune feature's availability in GCC High or DoD is uncertain, **verify for GCC High** against Microsoft's current US-sovereign-cloud service availability documentation before relying on it in your standards.

Requirements

Confirm every item below **before** you begin Section 3. The **category-level global requirements** (Entra tenant, Intune subscription, enrolled devices, supported Windows baseline, network access) are listed in `global-requirements-intune-solutions.md` and are assumed here. This file covers the requirements specific to standing up an **application management governance and operating model**.

Each requirement is written as **Description / Why required / How to verify / How to implement if missing.** Do not assume any of them already exist — several are organisational rather than technical and

are the most common reason this kind of programme stalls.

> ***Government tenants:*** *wherever a URL appears below, substitute the sovereign equivalent —* `https://endpoint.microsoft.us` *for Intune,* `https://portal.azure.us` *for Entra/Azure,* `https://login.microsoftonline.us` *for sign-in, and* `https://graph.microsoft.us` *for Graph. The steps themselves are unchanged.*

R1. A named executive sponsor with authority over application decisions

Description: A single named executive — typically the CIO, IT Director, or Head of Digital Workplace — who publicly owns the operating model and will uphold an Application Review Board decision when it is challenged.

Why required: The Application Review Board's decision rights are the core of this model. Without a sponsor who will back a rejection, the first time a senior stakeholder is told "no" the board becomes advisory, and the model collapses into a request queue. This is the single most common failure mode of application governance programmes, and it is an organisational prerequisite, not a technical one.

How to verify: You can name the individual, and they have confirmed in writing (email is sufficient) that (a) they endorse the operating model and (b) escalations of a board decision come to them. If you cannot produce that email, this requirement is **not met**.

How to implement if missing: Prepare a one-page brief using `executive-summary.md` from this package, present the current state (use the output of `APPOPS-Discovery.ps1` — the count of unowned and orphaned applications is usually persuasive on its own), and ask explicitly for the sponsorship and the escalation path. **Do not begin Step 3 (standing up the board) until this is secured.**

R2. Intune Administrator role (for the initial build)

Description: The **Intune Administrator** Entra role, held by the person performing the configuration steps in Section 3.

Why required: Steps 4, 5, 6, and 8 create Intune scope tags, a custom Intune role, role assignments, and app metadata changes. Scope tag and role creation require Intune Administrator; they are not available to Help Desk Operator or to a read-only role.

How to verify: Sign in to `https://intune.microsoft.com`, then navigate to **Tenant administration** → **Roles** → **All roles**. If the blade loads and you can open **Create** on the **Scope (Tags)** page (**Tenant administration** → **Roles** → **Scope (Tags)**), you hold sufficient rights.

How to implement if missing: Have a Global Administrator navigate to `https://entra.microsoft.com` → **Identity** → **Roles & admins** → **Roles & admins** → search for and select **Intune Administrator** → **Add assignments** → select your account → **Add**. Prefer a **time-bound eligible** assignment through Privileged Identity Management where licensed; the build work takes hours, not permanent access.

R3. Least-privilege operating roles (the steady-state model)

Description: After the build, day-to-day application work must **not** run on Intune Administrator. You need the ability to create a **custom Intune role** — named `Role – App Deployment Engineer` in this solution — granting only Managed apps **Read**, **Create**, **Update**, **Delete**, and **Assign**, scoped by scope tag.

Why required: Governance that grants every packaging engineer full Intune Administrator is not governance. The custom role is what makes the deployment engineer accountability in the RACI real: the engineer can change applications and nothing else, and only within the scope tags they are assigned. It also means an audit can show that application change rights are separable from device, compliance, and Conditional Access rights.

How to verify: In Intune, navigate to **Tenant administration** → **Roles** → **All roles** → **Create**. If the **Add Custom Role** wizard opens and lets you set permissions, you can create the role. Then check **Tenant administration** → **Roles** → **All roles** and confirm whether `Role – App Deployment Engineer` already exists (it will not, on a first run).

How to implement if missing: Section 3, Step 5 creates the role from scratch, click by click. The only prerequisite is R2.

R4. Ability to create Microsoft Entra security groups

Description: Rights to create Entra ID security groups, and — critically — confirmation that the tenant's **group creation restriction** setting does not block you.

Why required: The model creates seven named security groups (four role groups, three deployment ring groups) in Step 4 and Step 7. Many tenants restrict group creation to a specific group of users via the Entra **Groups** → **General** → **Security group creation** setting; if you are not in that allowed set, every group-creation step will fail with a permissions error that does not obviously say why.

How to verify: Navigate to `https://entra.microsoft.com` → **Identity** → **Groups** → **All groups** → **New group**. If the **New Group** blade opens and the **Create** button is enabled after you fill the required fields, you can create groups. Also check **Identity** → **Groups** → **General** and read the value of **Users can create security groups in Azure portals, API or PowerShell** — if it is **No**, you must be in the delegated group or hold a directory role.

How to implement if missing: Have a Global Administrator or Groups Administrator either (a) assign you the **Groups Administrator** Entra role, or (b) add your account to the group nominated in **Identity** → **Groups** → **General** → **Security group creation**. Groups Administrator is the least- privilege option and is sufficient for everything in this package.

R5. Microsoft Graph PowerShell SDK and the required delegated scopes

Description: A workstation with Windows PowerShell 5.1 or PowerShell 7 and the **Microsoft.Graph** module installed, plus consent to the delegated Graph permissions the five scripts require:

Script	Scopes required
`APPOPS-Discovery.ps1`	`DeviceManagementApps.Read.All`, `DeviceManagementRBAC.Read.All`, `Group.Read.All`

Script	Scopes required
`APPOPS-Validation.ps1`	`DeviceManagementApps.Read.All`, `DeviceManagementRBAC.Read.All`
`APPOPS-Report.ps1`	`DeviceManagementApps.Read.All`, `DeviceManagementRBAC.Read.All`, `Group.Read.All`
`APPOPS-Export-ApprovalEvidence.ps1`	`DeviceManagementApps.Read.All`, `DeviceManagementApps.Read.All`, `DeviceManagementConfiguration.Read.All`
`APPOPS-Remediation.ps1`	`DeviceManagementApps.ReadWrite.All`, `DeviceManagementRBAC.Read.All`

Four of the five scripts are **strictly read-only**. Only `APPOPS-Remediation.ps1` writes, and it defaults to `-WhatIf`.

Why required: The scripts read the application estate, assignments, scope tags, role definitions, and audit events through Microsoft Graph. Without the module and consent, they return nothing or fail at `Connect-MgGraph`.

How to verify: Run `$PSVersionTable.PSVersion` (expect `5.1.x` or `7.x`), then `Get-Module Microsoft.Graph -ListAvailable` (expect at least one version listed). Then run `APPOPS-Discovery.ps1` — at the first `Connect-MgGraph` prompt, sign in and consent. If it returns a table of applications, access is working. You can confirm granted permissions afterwards at `https://entra.microsoft.com` → **Identity** → **Applications** → **Enterprise applications** → search **Microsoft Graph Command Line Tools** → **Security** → **Permissions**.

How to implement if missing: Install the SDK with `Install-Module Microsoft.Graph -Scope CurrentUser -Force`. Run scripts with `powershell -ExecutionPolicy Bypass -File .\APPOPS-Discovery.ps1`. If your tenant requires admin consent for the **Microsoft Graph Command Line Tools** application, a Global Administrator must grant it once at Enterprise applications → the app → **Security** → **Permissions** → **Grant admin consent**.

R6. A system of record for the Application Register

Description: A durable, access-controlled place to hold the **Application Register**, the **ARB Decision Records**, and the **Application Runbooks**. Acceptable options, in descending order of preference: your ITSM/CMDB configuration item class; a SharePoint Online list; a Microsoft Lists list; a governed Excel workbook in SharePoint with version history enabled.

Why required: Intune holds *technical* facts about applications. It does not hold business ownership, licence entitlement position, review dates, or approval evidence, and its Notes field is not a substitute for a register that can be reported on and audited. Without a system of record the model has nowhere to store its own output.

How to verify: You can name the specific list/table/CI class, you can create items in it, and it has (a) version history or an audit trail and (b) access control such that Application Owners can read and only the register custodian can edit key fields.

How to implement if missing: The fastest acceptable option is a SharePoint Online list. Navigate to your IT team's SharePoint site → **New** → **List** → **Blank list** → name it `Application Register` → **Create**. Section 3, Step 2 defines the exact columns. Enable version history at **Settings** → **List settings** → **Versioning settings** → **Create a version each time you edit an item in this list** → **Yes**.

R7. An existing change management process (or a decision that none exists)

Description: Knowledge of whether the organisation operates a formal **Change Advisory Board (CAB)** or equivalent change control, and if so, its cadence, its lead time, and what it classifies as a standard/normal/emergency change.

Why required: Step 8 defines the **interface** between application change and enterprise change control. If a CAB exists and you do not integrate with it, application changes will either bypass change control (an audit finding) or clog it with routine version updates (a political problem). If no CAB exists, the model's risk tiers become the change process itself, which changes how Step 8 is implemented.

How to verify: You can state the CAB's meeting cadence, its submission deadline, and where standard/pre-approved changes are recorded. Ask your Service Management or IT Operations lead directly; do not infer this from a wiki page.

How to implement if missing: If no CAB exists, that is a valid answer — record it, and implement Step 8 with the Application Review Board holding change authority for all four risk tiers. If a CAB exists but is undocumented, get the cadence and standard-change definition in writing from the process owner before drafting the change matrix.

R8. An application discovery data source beyond Intune

Description: At least one source that shows what is actually *installed on endpoints*, as opposed to what Intune deploys. Options: **Microsoft Defender for Endpoint** software inventory (**Vulnerability management** → **Software inventory** in the Defender portal), Intune **Discovered apps** (**Apps** → **Monitor** → **Discovered apps**), or a third-party asset management tool.

Why required: The Application Register must reconcile against reality. Applications present on endpoints but absent from Intune and from the Register are ungoverned by definition — they were installed by users, by legacy imaging, or by a departed process. Step 2 cannot produce a truthful register without this reconciliation input, and the "shadow application" count is one of the scorecard's most useful risk metrics.

How to verify: In Intune, navigate to **Apps** → **Monitor** → **Discovered apps**. If the list populates with applications and device counts, you have a usable source. If you have Defender for Endpoint, also confirm **Vulnerability management** → **Software inventory** loads in the Defender portal.

How to implement if missing: Intune Discovered apps requires no additional configuration for enrolled devices — if it is empty, confirm devices have checked in recently (**Devices** → **All devices**, review **Last check-in**). If you have no Defender for Endpoint licence and Discovered apps is unavailable, record this as a known gap in the register's accuracy and plan a manual sampling exercise instead; do not skip the reconciliation entirely.

R9. Nominated individuals for the four RACI roles

Description: Named people (not job titles, not teams) willing to hold each of the four accountabilities for the pilot scope: **Application Owner**, **Technical Owner**, **Security Reviewer**, **Deployment Engineer**.

Why required: Step 1 populates the RACI, and Step 4 puts these people into named Entra security groups. An unfilled role means the corresponding gate in the model has no owner, which in practice means it is skipped. Security Reviewer is the role most often left unnamed and the one whose absence does the most damage.

How to verify: You have a list with four names (or four sets of names, where a role is shared) and each individual has been asked and agreed. Their manager is aware of the time commitment — a realistic figure for the ARB is 60–90 minutes per cycle plus review preparation.

How to implement if missing: Take the gap list to the executive sponsor from R1. The Security Reviewer must come from Information Security, not from IT operations — self-review is not review. Where an organisation genuinely has no security function, the model still works, but record the compensating control (for example, an external annual assessment) explicitly in the standards document rather than pretending the gate exists.

R10. Licence and entitlement data access

Description: Access to the organisation's software licence position — the contracts, entitlement counts, and renewal dates — held by Procurement, Finance, or a Software Asset Management function.

Why required: The intake workflow's licensing gate (Step 3) and the standards document's entitlement criterion (Step 6) both require somebody to answer "do we have the right to deploy this to N devices?" If nobody can answer, the gate is theatre and the organisation carries unquantified audit exposure from its own deployments.

How to verify: You can name the person or system that holds the entitlement record, and you have successfully retrieved the entitlement position for one existing application as a test.

How to implement if missing: Identify the contract owner in Procurement and agree a service — even an informal one — where an intake request can be checked for entitlement within the intake SLA defined in Step 3. Record the agreed turnaround in the Application Standards Document. If no such function exists, assign the entitlement check to the Application Owner and require them to attach the licence agreement to the intake request.

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an

administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

Government tenants: substitute the US sovereign portal URLs — `endpoint.microsoft.us`, `portal.azure.us`, `login.microsoftonline.us`, `graph.microsoft.us`. Where a feature's GCC High / DoD availability is uncertain, verify against Microsoft's current US-sovereign-cloud service availability documentation.

Implementation

This section stands up the operating model end to end. Work through the steps **in order** — later steps depend on objects created by earlier ones. Every object you create is given an **exact name**; use that name verbatim, because Section 5 (Testing), Section 6 (Troubleshooting), and all five PowerShell scripts refer to these names directly.

Naming convention used throughout: `<Purpose> – <Scope> – <Stage>`, using an en dash (–) surrounded by single spaces. Keep it consistent; the validation script matches on it.

> ***Government tenants:*** *substitute* `https://endpoint.microsoft.us` *for* `https://intune.microsoft.com` *and* `https://portal.azure.us` *for* `https://entra.microsoft.com` *in every navigation path below. The blade names, tab names, and field names are the same.*

Total elapsed time: expect 3–5 working days of effort for Steps 1–8, then two full board cycles (typically 4–6 weeks) for the pilot in Step 13.

Step 1 — Baseline the current application estate

You cannot govern what you have not counted. Produce the "before" picture first; it is also the evidence you will use to hold the executive sponsor's attention.

1. On your admin workstation, open **PowerShell** as your normal (non-elevated) user.
2. Install the Graph SDK if it is not present:

```
Install-Module Microsoft.Graph -Scope CurrentUser -Force
```

1. Change to the folder containing this package's scripts.
2. Run the discovery script, exporting to CSV:

```
powershell -ExecutionPolicy Bypass -File .\APPOPS-Discovery.ps1 -CsvPath .\AppEstate-Baseline.csv
```

For a Government tenant add `-GraphEnvironment USGov`.

1. At the `Connect-MgGraph` prompt, sign in with an account holding the rights from R2 and **Accept** the consent prompt.

1. Open `AppEstate-Baseline.csv` in Excel. Record these five numbers — they are your baseline scorecard, and Step 13 will measure against them:

Metric	Where it comes from
Total applications in Intune	Row count
Applications with **no assignment** (orphaned)	`AssignmentCount` = 0
Applications assigned to **All Users** or **All Devices**	`TargetsAllUsers` or `TargetsAllDevices` = `True`
Applications with **no owner recorded**	`GovernedOwner` is blank
Applications with **no scope tag** other than Default	`ScopeTags` = `Default`

1. Now capture what is installed but **not** deployed by Intune. In Intune, navigate to

Apps → **Monitor** → **Discovered apps**. Click **Export** → **Yes** to generate the CSV, then download it when the notification appears.

1. In Excel, compare the Discovered apps list against the Intune-deployed list from step 6. Count the

applications present on endpoints but **absent from Intune**. Record this as **Shadow applications** — it is usually the most uncomfortable and most useful number in the baseline.

1. Save all three files to your system of record (R6) in a folder named `Application Governance –

Baseline – <YYYY-MM>`. Do not skip this; Step 13 compares against it.

Step 2 — Create the Application Register

The Application Register is the authoritative record. Intune is the *technical* system of record; the Register is the *governance* system of record.

1. Open your system of record from R6. These instructions use SharePoint Online; adapt the field

types if you are using an ITSM CI class.

1. Navigate to your IT team's SharePoint site → **New** → **List** → **Blank list**.
2. In **Name**, enter exactly `Application Register`. In **Description**, enter

`Authoritative record of every application in the managed estate, its owners, approval status, and lifecycle state.` Leave **Show in site navigation** ticked. Click **Create**.

1. Add the following columns. For each, click **+ Add column**, choose the type shown, enter the name

exactly as written, then click **Save**.

Column name	Type	Notes
`Application Name`	Single line of text	Rename the default `Title` column to this.
`Vendor`	Single line of text	
`Version`	Single line of text	Current approved version.
`Application Owner`	Person	Business accountability. Required.
`Technical Owner`	Person	Package and runbook accountability. Required.
`Security Reviewer`	Person	Who signed the risk assessment.
`Deployment Engineer`	Person	Who executes changes.
`Data Classification`	Choice	`Public`, `Internal`, `Confidential`, `Restricted`.
`Business Justification`	Multiple lines of text	Copied from the intake request.

Column name	Type	Notes
Licence Position	Choice	Entitled, Entitlement pending, Free/no licence, Non-compliant.
Entitlement Count	Number	Seats or devices licensed.
Lifecycle State	Choice	Requested, In review, Approved – pilot, Approved – production, Deprecated, Retired.
ARB Decision Reference	Single line of text	Format ARB-<YYYY>-<NNN>.
Approval Date	Date and time	Date only.
Next Review Date	Date and time	Approval date + review interval (Step 6).
Intune App Id	Single line of text	The GUID from Intune; the join key to the scripts.
Risk Tier	Choice	Tier 1 – Low, Tier 2 – Moderate, Tier 3 – High, Tier 4 – Critical.
Runbook Link	Hyperlink	Points at the Application Runbook.
Support Tier	Choice	L1 resolvable, L2 required, L3/vendor required.

1. Enable version history: click the **gear icon** (top right) → **List settings** → **Versioning settings → **set** Create a version each time you edit an item in this list **to** Yes → OK**.

1. Set permissions: **gear icon → List settings → Permissions for this list → Stop Inheriting Permissions → OK**. Then** Grant Permissions **and give the register custodian** Edit**, and all Application Owners** Read**. Approval fields must not be editable by requesters.

1. Import your Step 1 baseline. Open AppEstate-Baseline.csv, and for each application create a register item populating Application Name, Vendor, and Intune App Id. Leave the ownership and approval fields blank — the gaps are the point, and Step 12 will fill them.

1. Create a second list named exactly ARB Decision Record with these columns: Decision Reference (single line of text, format ARB-<YYYY>-<NNN>), Application Name (single line of text), Decision Date (date), Decision (choice: Approved, Approved with conditions, Deferred – evidence required, Rejected), Conditions (multiple lines of text), Members Present (multiple lines of text), Evidence Pack Link (hyperlink), Decision Rationale (multiple lines of text). Enable version history as in step 5.

Step 3 — Define the four accountabilities (the RACI)

1. Create a document in your system of record named exactly `Application Management RACI – Enterprise – v1.0`.

1. Populate it with the following matrix. This is the model's authority definition; adapt names, not structure.

Activity	Application Owner	Technical Owner	Security Reviewer	Deployment Engineer	ARB
Raise intake request	**A/R**	C	I	I	I
Provide business justification	**A/R**	I	I	I	C
Confirm licence entitlement	**A**	I	I	I	C
Perform security review	C	C	**A/R**	I	C
Assess technical supportability	C	**A/R**	C	C	C
Approve for production	C	C	C	I	**A/R**
Package and configure the application	I	**A**	I	**R**	I
Classify a change's risk tier	C	**A/R**	C	C	I
Execute a ringed deployment	I	**A**	I	**A/R**	I
Authorise a rollback	C	**A**	I	**R**	I
Own the Application Runbook	I	**A/R**	I	C	I
Answer L2 support escalations	I	**A/R**	I	C	I
Approve retirement	**A/R**	C	C	I	**A**

Activity	Application Owner	Technical Owner	Security Reviewer	Deployment Engineer	ARB
Sign off the periodic review	**A/R**	C	**C**	I	**A**

A = Accountable (one only), R = Responsible, C = Consulted, I = Informed.

1. Immediately below the matrix, record **the four escalation rules** verbatim:

• A request with no named **Application Owner** is returned to the requester, not queued.

• A change with no named **Technical Owner** cannot be approved at any risk tier.

• An application handling `Confidential` or `Restricted` data with no completed **Security Reviewer** sign-off cannot reach `Approved – production`.

• A rollback may be authorised by the **Deployment Engineer** without further approval; it is always a safe action and must never require a meeting.

1. Have the executive sponsor from R1 approve the document, and record the approval date in it.

Step 4 — Create the four role security groups in Entra ID

These groups make the RACI enforceable — they are what the Intune custom role and the ARB invitation are bound to.

1. Navigate to `https://entra.microsoft.com` and sign in with an account holding the rights from R4.
2. In the left navigation, expand **Identity**, then click **Groups** → **All groups**.
3. Click **+ New group** at the top of the blade.
4. Complete the **New Group** form for the first group:

• **Group type:** `Security`

• **Group name:** `APPGOV – Application Owners`

• **Group description:** `Business owners accountable for applications in the managed estate. Membership is maintained by the register custodian.`

• **Microsoft Entra roles can be assigned to the group:** `No`

• **Membership type:** `Assigned`

• **Owners:** add the register custodian.

• **Members:** add the individuals nominated in R9 for this role.

1. Click **Create**.
2. Repeat steps 3–5 exactly three more times, creating:

Group name	Membership type	Description
APPGOV – Technical Owners	Assigned	Engineers accountable for application packages, configuration, and runbooks.
APPGOV – Security Reviewers	Assigned	Information Security staff who perform and sign application risk assessments.
APPGOV – Deployment Engineers	Assigned	Operators authorised to execute approved application changes in Intune.

1. **Use** `Assigned`**, not** `Dynamic`**, for all four.** These are accountability groups whose membership is a deliberate governance decision; a dynamic rule would let an HR attribute change silently grant or remove application change rights.

1. Verify: still in **Identity** → **Groups** → **All groups**, type APPGOV in the search box. Exactly four groups must be listed. Record each group's **Object Id** (open the group → **Overview** → copy **Object Id**) into the RACI document from Step 3.

Step 5 — Create the scope tags and the least-privilege Intune role

This is the step that converts "the deployment engineers own application changes" from a sentence into a permission boundary.

5a — Create the two scope tags

1. Navigate to `https://intune.microsoft.com`.
2. In the left navigation, click **Tenant administration**, then **Roles**, then **Scope (Tags)**.
3. Click **+ Create**.
4. On the **Basics** tab, enter:

• **Name:** `Scope – App Management – Prod`

• **Description:** `Applications approved by the Application Review Board for production deployment.`

Click **Next**.

1. On the **Assignments** tab, click **+ Add groups**, select APPGOV – Deployment Engineers, and click **Select**. Click **Next**.

1. On the **Review + create** tab, confirm the name reads exactly `Scope – App Management – Prod`,

then click **Create**.

1. Repeat steps 3–6 to create a second scope tag:

• **Name:** `Scope – App Management – Pilot`

• **Description:** `Applications in pilot or under review; not yet approved for production.`

• **Assignments:** `APPGOV – Deployment Engineers`

5b — Create the custom Intune role

1. Still in Intune, navigate to **Tenant administration** → **Roles** → **All roles**.

2. Click **+ Create**, then select **Windows role** (this is the standard custom-role wizard;

despite the label, the permissions you select govern the resources chosen, and Managed apps permissions are platform-independent).

1. On the **Basics** tab:

• **Name:** `Role – App Deployment Engineer`

• **Description:** `Least-privilege role permitting application creation, update, deletion, and assignment within assigned scope tags only. Does not grant device, compliance, or Conditional Access rights.`

Click **Next**.

1. On the **Permissions** tab, expand **Managed apps** and set:

• **Read** → `Yes`

• **Create** → `Yes`

• **Update** → `Yes`

• **Delete** → `Yes`

• **Assign** → `Yes`

Then expand **Organization** and set **Read** → `Yes` (required for the console to render). **Leave every other permission category set to** `No`**.** In particular do not grant **Device configurations**, **Device compliance policies**, **Security baselines**, or **Remote tasks**. Click **Next**.

1. On the **Scope tags** tab, click **+ Select scope tags**, tick `Scope – App Management – Prod` and `Scope – App Management – Pilot`, and click **Select**. Click **Next**.

1. On the **Review + create** tab, confirm the name and permissions, then click **Create**.

5c — Assign the role

1. Navigate to **Tenant administration** → **Roles** → **All roles** and click `Role – App Deployment Engineer`.

1. In the role's left menu click **Assignments**, then click **+ Assign**.

2. On the **Basics** tab, enter **Assignment name:** `Assign – App Deployment Engineers – Prod`, and

an optional description. Click **Next**.

1. On the **Admin Groups** tab, click **+ Select groups to include**, choose

`APPGOV – Deployment Engineers`, and click **Select**. Click **Next**.

1. On the **Scope Groups** tab, choose the scope of devices/users these admins may act on. For a

first deployment select **All devices** and **All users** unless you are delegating by business unit. Click **Next**.

1. On the **Scope tags** tab, click **+ Select scope tags** and tick both

`Scope – App Management – Prod` and `Scope – App Management – Pilot`. Click **Next**.

1. On the **Review + create** tab, click **Create**.

2. **Now remove the broad grants.** Navigate to `https://entra.microsoft.com` → **Identity → Roles &

admins → Roles & admins → Intune Administrator → Assignments**. Review every standing assignment. Anyone whose only need is application work should be removed here and left with** `Role – App Deployment Engineer` **instead. Do this** after** confirming the new role works (Section 5, Gate 3) — never before.

Step 6 — Create the deployment ring groups

Ringed deployment is the change-management control that makes rollback cheap. No application may be assigned directly to *All Users* or *All Devices* under this model.

1. Navigate to `https://entra.microsoft.com` → **Identity** → **Groups** → **All groups** → **+ New group**.

2. Create the first ring:

• **Group type:** `Security`

• **Group name:** `PILOT – App Ring 1 – Validation`

• **Group description:** `Ring 1. IT and technical validators. First recipients of every approved application change. Target size 1–2% of the estate.`

• **Membership type:** `Assigned`

• **Members:** the endpoint engineering team and the Technical Owners.

Click **Create**.

1. Repeat for the remaining two rings:

Group name	Membership type	Intended population
`PILOT – App Ring 2 – Early Adopters`	`Assigned`	Volunteer business users across every major function. Target 5–10% of the estate.
`PROD – App Ring 3 – Broad`	`Dynamic User` or `Assigned`	The remainder of the estate.

1. If you make Ring 3 dynamic, on the **New Group** blade set **Membership type** to `Dynamic User`,

click **Add dynamic query**, switch to the **Rule syntax** editor, and enter a rule that excludes the earlier rings — for example:

```
(user.accountEnabled -eq true) and (user.userType -eq "Member")
```

then manage Ring 1 and Ring 2 exclusion at the **assignment** level in Intune (each app's assignment lets you add an **excluded group**). Click **Save**, then **Create**.

1. Record all three group **Object Ids** in the RACI document.
2. Define and publish the **ring dwell times** — the minimum time an application must sit in a ring before promotion. Add this table to the standards document you create in Step 7:

From → To	Minimum dwell	Promotion gate
Ring 1 → Ring 2	3 business days	Zero Sev-1/Sev-2 incidents; install success ≥ 95%.
Ring 2 → Ring 3	5 business days	Zero Sev-1 incidents; install success ≥ 98%; no open licence question.

Step 7 — Publish the Application Standards Document

This is the document that makes "approved" a test rather than an opinion.

1. Create a document in your system of record named exactly `Application Standards – Enterprise – v1.0`.
1. Include the following six sections. Each must be written so that a reviewer can answer *yes* or *no* — avoid "should be considered".

Section 1 — Definition of approved. An application is `Approved – production` only when all of the following are true: a named Application Owner and Technical Owner exist in the register; the security review is complete and signed; the licence position is `Entitled` or `Free/no licence`; a runbook exists and is linked; the package conforms to the naming standard; and an ARB Decision Record exists with a decision of `Approved` or `Approved with conditions` (with all conditions met).

Section 2 — Security review criteria. The Security Reviewer must record a determination against each of: data classification handled; whether the application transmits data outside the tenant boundary and to where; authentication method (SSO-capable vs local credentials); whether it requires local administrator rights to run; its network egress requirements; vendor security posture evidence (a current SOC 2 / ISO 27001 attestation or equivalent, or a documented exception); known CVEs at the proposed version; and whether it requires an App Protection Policy or an application-control allow-list entry.

Section 3 — Licensing and entitlement check. Record the licence model (per-user, per-device, site, free, open-source), the entitlement count held, the count required by the proposed assignment, the contract owner, and the renewal date. A proposed deployment that exceeds the entitlement count is `Non-compliant` and cannot be approved.

Section 4 — Naming standard. Every application object in Intune must be named:

```
<Vendor> <Product> <Version> – <Platform> – <Ring>
```

For example: `Adobe Acrobat Reader 24 – Win – Ring1`, `Google Chrome 126 – Win – Ring3`, `Zoom Workplace 6.1 – macOS – Ring2`. Use the en dash with single spaces. `APPOPS-Validation.ps1` tests conformance to exactly this pattern.

Section 5 — Supportability bar. Before production approval an application must have: a documented install and uninstall method; a documented rollback method; a named L2 owner; a runbook covering the five most likely failure modes; and a vendor support contract or a documented acceptance that none exists.

Section 6 — Review interval. Set `Next Review Date` by risk tier: `Tier 4 – Critical` every 6 months; `Tier 3 – High` every 12 months; `Tier 1 – Low` and `Tier 2 – Moderate` every 24 months. A review that lapses moves the application to `Deprecated` until it is re-reviewed.

1. Have the executive sponsor and the Head of Information Security approve the document. Record both

approval dates.

1. Publish it where every requester can read it **before** raising a request. A standards document

nobody can find generates the same rework as no standards document.

Step 8 — Stand up the intake workflow

1. Create the intake form. Using Microsoft Forms: navigate to `https://forms.office.com` → **New

Form → **name it exactly** `Application Intake Request Form`**. Add these questions, all marked** Required** unless stated:

#	Question	Type
1	Application name, vendor, and version	Text
2	Named Application Owner (business accountability)	Text
3	Business justification — what problem does this solve, and what happens if we do not deploy it?	Long text
4	Has an existing approved application been evaluated for this need? Which, and why is it insufficient?	Long text
5	How many users or devices require it?	Number
6	Data classification handled	Choice: Public / Internal / Confidential / Restricted
7	Licence model and entitlement already held	Text

#	Question	Type
8	Contract or quote reference	Text (optional)
9	Required by date, and the driver for that date	Text
10	Platforms required	Choice, multiple: Windows / macOS / iOS / Android
11	Does it require local administrator rights to install or run?	Choice: Yes / No / Unknown
12	Vendor security documentation attached?	File upload (optional)

1. Route the responses. Open the form → **Responses** tab → the ■ menu → **Create a Flow** to

send each response into a SharePoint list named `Application Intake Queue`, or simply export responses to Excel weekly if you have no Power Automate licence. The mechanism matters far less than the discipline.

1. Define and publish the **intake SLA**: the register custodian triages every request within **3

business days** and returns it, tables it for the next ARB, or fast-tracks it under the Tier 1 pre-approval rule (Step 9).

1. Define the **return criteria** explicitly. A request is returned to the requester — not queued —

when it lacks a named Application Owner, a business justification, or a licence position. Publish these three so returns are not perceived as arbitrary.

Step 9 — Stand up the Application Review Board

1. Create a recurring Outlook meeting named exactly

`Application Review Board – Enterprise – Fortnightly`.

1. Set the cadence and duration:

• **Cadence:** every two weeks. Weekly becomes unsustainable; monthly makes IT the bottleneck and

drives shadow procurement.

• **Duration:** 60 minutes. If you consistently overrun, the intake gate in Step 8 is too loose.

1. Define **membership**. Invite as **required**: the register custodian (chair), one member of

`APPGOV – Technical Owners`, one member of `APPGOV – Security Reviewers`, and a Procurement or SAM representative. Invite as **optional**: the requesting Application Owner for their item only.

1. Define **quorum** and record it in the standards document: the chair, plus one Technical Owner,

plus one Security Reviewer. **Without a Security Reviewer present, no application handling `Confidential` or `Restricted` data may be approved** — such items are deferred, not waved through.

1. Define the four permitted **decisions**. The board must select exactly one per item:

• `Approved` — proceeds to Ring 1.

• `Approved with conditions` — proceeds, with the conditions recorded and a named owner and due

date for each.

• `Deferred – evidence required` — the specific missing evidence is named, along with who supplies

it and by when.

• `Rejected` — with a written rationale. A rejection must always state what would change the

answer.

1. Define the **evidence pack** required before an item may be tabled. The chair refuses to table

items lacking any of: the completed intake request; the security review determination against all nine criteria in Standards Section 2; the licence position; a proposed risk tier; and a named Technical Owner who has confirmed they will take the runbook.

1. Define the **Tier 1 pre-approval (standard change) rule** so the board is not consumed by routine

work: a version update to an already-approved application, with no change to data handling, no new network egress, and no licence change, is **pre-approved** and does not require an ARB slot. It is recorded in the ARB Decision Record retrospectively with the reference `ARB-<YYYY>-STD-<NNN>`. This one rule typically removes 60–80% of the board's volume.

1. Record every decision in the `ARB Decision Record` list from Step 2 within **1 business day** of

the meeting. Decisions not recorded within that window are treated as not made.

Step 10 — Publish the change classification matrix and the CAB interface

1. Create a document named exactly `Application Change Classification – Enterprise – v1.0`.

2. Populate it with the four risk tiers:

Risk tier	Definition	Approval required	Ring policy	Change window	Rollback authority
`Tier 1 – Low`	Version update to an approved app; no change to data handling, egress, or licence.	Pre-approved (standard change). Technical Owner confirms classification.	Ring 1 → Ring 2 → Ring 3, standard dwell.	Any business day, 09:00–16:00.	Deployment Engineer, unilaterally.

Risk tier	Definition	Approval required	Ring policy	Change window	Rollback authority
`Tier 2 – Moderate`	New app, Tier 1/2 data only; or a configuration change to an approved app.	ARB approval.	Full ring progression, standard dwell.	Any business day, 09:00–16:00.	Deployment Engineer, unilaterally.
`Tier 3 – High`	App handling `Confidential` data; or any change affecting more than 25% of the estate; or a supersedence that removes an existing app.	ARB approval **and** Security Reviewer sign-off.	Full ring progression, **doubled** dwell times.	Outside core hours; CAB notified.	Deployment Engineer, with Technical Owner informed within 1 hour.
`Tier 4 – Critical`	App handling `Restricted` data; or any change to a business-critical or clinical/regulated application; or a change requiring local admin rights.	ARB approval, Security Reviewer sign-off, **and** CAB approval.	Full ring progression with an explicit written go/no-go gate at each promotion.	Scheduled CAB window only.	Deployment Engineer executes; Technical Owner and CAB chair notified immediately.

1. Define the **CAB interface** using your answer to R7:

• If a CAB exists: Tier 1 and Tier 2 changes are registered as **standard/pre-approved** changes

and consume no CAB agenda time. Tier 3 changes are **notified** to CAB. Tier 4 changes are **submitted** to CAB by the change submission deadline, with the ARB Decision Reference quoted as the technical approval evidence. Record the CAB's cadence and deadline in this document.

• If no CAB exists: the ARB holds change authority for all four tiers. Record that explicitly so

an auditor is not left inferring it.

1. State the **rollback principle** prominently: *a rollback never requires approval.* Any Deployment

Engineer may roll back any application change at any time. Requiring approval to roll back converts a five-minute recovery into a two-hour outage, and it is the most common self-inflicted wound in application change management.

Step 11 — Define the support model and the runbook standard

1. Create a document named exactly `Application Support Matrix – Enterprise – v1.0`.

2. Define the tier boundaries in terms of *actions*, not seniority — this is what makes escalation decidable at 3am:

Tier	Owns	May do	Escalates when
L1 — Service Desk	First contact, triage, known-error resolution.	Confirm the user is in the correct ring group; confirm the app shows as assigned in the Company Portal; trigger a device sync; walk the user through the documented known-error steps in the runbook.	The app is assigned but not installing; the runbook's known-error steps do not resolve it; or the fault affects more than 3 users.
L2 — Endpoint Engineering	Package and deployment faults.	Read Intune install/detection state; inspect device logs; correct assignments; re-issue the package; execute an approved rollback.	The fault is in the application itself rather than its delivery; a code-level defect is suspected; or a licence/entitlement block is found.
L3 — Vendor / Application Specialist	The application's own behaviour.	Raise and manage the vendor case; test vendor-supplied fixes in Ring 1.	Never — L3 owns to resolution, with the Technical Owner accountable for the case.

1. Define **response and resolution targets** per severity and publish them:

Severity	Definition	Response	Resolution target
Sev 1	A business-critical application is unavailable to a whole site or function.	15 minutes	4 hours (or rollback)
Sev 2	An approved application fails to install or run for more than 10 users.	1 hour	1 business day
Sev 3	Individual user cannot access an approved application.	4 hours	3 business days
Sev 4	Request, query, or cosmetic issue.	1 business day	10 business days

1. Create the runbook template. Name it exactly `Application Runbook – <Application Name> – v1.0`

and require these seven sections: (1) application identity — vendor, version, Intune App Id, and register link; (2) named owners for all four RACI roles; (3) install and uninstall method; (4) detection/verification method — how to confirm it is genuinely present and working; (5) the five most likely failure modes with their known-error resolutions, written for L1; (6) the rollback procedure with expected duration; (7) vendor support contact, contract reference, and entitlement.

1. Make the runbook a **production gate**: an application cannot move to `Approved – production`

without a linked runbook. Enforce this in the register by requiring `Runbook Link` to be populated before `Lifecycle State` may be set to `Approved – production`.

Step 12 — Stamp governance metadata onto the existing estate

Now make the governance visible inside Intune itself, so an engineer looking at an app can see its governance state without opening the register.

1. Open `AppEstate-Baseline.csv` from Step 1 in Excel.

2. Add three columns: `GovernedOwner`, `ArbReference`, and `ScopeTagToApply`. For each application,

populate the Technical Owner's user principal name, the ARB decision reference (use `ARB-<YYYY>-LEGACY-<NNN>` for pre-existing applications ratified in bulk), and either `Scope – App Management – Prod` or `Scope – App Management – Pilot`.

1. Save the file as `AppEstate-Governance.csv`.

2. **Preview** the changes — the script defaults to `-WhatIf` and writes nothing:

```
powershell -ExecutionPolicy Bypass -File .\APPOPS-Remediation.ps1 -InputCsv .\AppEstate-Governance.csv
```

1. Read the preview output carefully. Confirm the app names and the values match your intent.

2. Apply the changes by re-running with `-Apply`:

```
powershell -ExecutionPolicy Bypass -File .\APPOPS-Remediation.ps1 -InputCsv .\AppEstate-Governance.csv
```

The script writes a structured governance block into each application's **Notes** field and applies the named scope tag. It does not change assignments, packages, or any deployment behaviour.

1. Spot-check in the console: navigate to **Apps** → **All apps**, open any application you stamped, and

click **Properties**. The **Notes** field must show the governance block, and **Scope tags** must show the tag you applied.

1. Rename any application that does not conform to the Step 7 naming standard. Do this in

Apps → **All apps** → **<app>** → **Properties** → **Edit** (next to **App information**) → change **Name** → **Review + save**. Renaming an app does not affect existing installs or assignments.

Step 13 — Pilot the model for two full board cycles

Do not roll the model across the whole estate immediately. Run it narrow and prove it.

1. Choose the pilot scope: **one business function** (for example, Finance) **or one application

family** (for example, all creative-suite applications). Record the choice.

1. Run the model exactly as documented for **two complete ARB cycles** — with a fortnightly cadence, four weeks.

1. During the pilot, route **every** new or changed application in scope through the intake form. No exceptions, including urgent ones — an urgent request is a Tier 3 or Tier 4 change, not a bypass.

1. At the end of each cycle, run the validation and reporting scripts:

```
powershell -ExecutionPolicy Bypass -File .\APPOPS-Validation.ps1 -CsvPath .\Validation-Cycle1.csv
powershell -ExecutionPolicy Bypass -File .\APPOPS-Report.ps1 -CsvPath .\Scorecard-Cycle1.csv
```

1. Hold a 30-minute retrospective after cycle 2 with the board members. Ask three questions: which gate produced the most rework, which evidence requirement was hardest to supply, and where did somebody attempt a bypass? Amend the standards document accordingly and increment its version.

1. Only when Section 5's gates pass, proceed to Step 14.

Step 14 — Expand to the whole estate and enter steady state

1. Extend the intake form as the **only** route for new applications tenant-wide. Announce the date, and have the executive sponsor from R1 send the announcement — not IT.

1. Bulk-ratify the remaining legacy estate. For each application still lacking owners, the register custodian tables a batch at the ARB for ratification with `ARB-<YYYY>-LEGACY-<NNN>` references. Applications for which **no owner can be found** are moved to `Deprecated`, communicated with a 60-day notice, and then retired. This is the step that actually shrinks the estate.

1. Establish the steady-state cadence:

Activity	Frequency	Owner
Application Review Board	Fortnightly	Register custodian (chair)
Governance scorecard published (`APPOPS-Report.ps1`)	Monthly	Register custodian
Conformance validation run (`APPOPS-Validation.ps1`)	Monthly	Deployment Engineer
Discovered-apps reconciliation against the register	Quarterly	Technical Owners
Overdue review sweep (`Next Review Date` in the past)	Monthly	Register custodian
Standards document review	Annually	Executive sponsor + Security

1. Add the scorecard to an existing governance forum's standing agenda — an IT operations review or a service management board. Metrics reported nowhere change nothing.

1. Archive the Step 1 baseline alongside the first steady-state scorecard. The delta between them is

the business case for having done this, and it is what you present at the annual review.

Testing & Validation

The operating model is proven not by whether the documents exist, but by whether the **gates hold** under real traffic. This section defines the pass/fail tests. Run the structural gates (1–4) after Section 3's build, and the behavioural gates (5–8) during the Step 13 pilot.

Each gate has an explicit **PASS** condition. A gate that does not pass is not "mostly done" — it is a hole in the model, and the section it belongs to must be corrected before you expand.

> ***Government tenants:*** *the console URLs in the manual gates are* `endpoint.microsoft.us` *and* `portal.azure.us`*; add* `-GraphEnvironment USGov` *to every script invocation.*

Gate 1 — The structural objects exist (automated)

Test: Run the validation script.

```
powershell -ExecutionPolicy Bypass -File .\APPOPS-Validation.ps1 -CsvPath .\Validation.csv
```

PASS when every one of these lines reports PASS:

- Scope tag: `Scope – App Management – Prod`
- Scope tag: `Scope – App Management – Pilot`
- Custom role: `Role – App Deployment Engineer`
- Group: `APPGOV – Application Owners`, `APPGOV – Technical Owners`, `APPGOV – Security Reviewers`,

`APPGOV – Deployment Engineers`

- Group: `PILOT – App Ring 1 – Validation`, `PILOT – App Ring 2 – Early Adopters`,

`PROD – App Ring 3 – Broad`

If it fails: the named object is missing. Return to the step that creates it (Step 4 for APPGOV groups, Step 5 for scope tags and the role, Step 6 for ring groups) and confirm you used the name **verbatim**, including the en dash and single spaces.

Gate 2 — The custom role grants exactly the intended rights (manual)

Test: Sign in to the console **as a member of** `APPGOV – Deployment Engineers` (not as an Intune Administrator). Attempt each action:

Action	Expected
Open **Apps** → **All apps** and edit an app carrying a `Scope – App Management – *` tag.	**Allowed.**
Open **Devices** → **Configuration** and try to create a configuration profile.	**Blocked / not visible.**

Action	Expected
Open **Endpoint security** → **Conditional Access**.	**Blocked / not visible.**
Edit an app that carries **only** the Default scope tag.	**Blocked** (out of the role's scope).

PASS when the deployment engineer can change **applications within the scope tags** and can do **nothing else**. If they can edit device configuration or compliance, the role in Step 5b has excess permissions — remove them.

Gate 3 — Broad grants have been removed (manual)

Test: Navigate to `https://entra.microsoft.com` → **Identity** → **Roles & admins** → **Roles & admins** → **Intune Administrator** → **Assignments**.

PASS when the only standing Intune Administrator assignments are the platform administrators who genuinely need tenant-wide rights. Anyone whose work is *application deployment only* must have been moved to `Role – App Deployment Engineer` (Step 5c, step 8). A long list of standing Intune Administrators is a finding, not a pass.

Gate 4 — Every application is governed (automated)

Test: Run the discovery script and read the governance baseline block.

```
powershell -ExecutionPolicy Bypass -File .\APPOPS-Discovery.ps1
```

PASS when, after Step 12, all four counts read **0**:

- Orphaned (no assignment): `0`
- All Users/All Devices: `0`
- No governed owner: `0`
- No scope tag (Default): `0`

A residual count is the exact list of applications still to remediate. Zero orphaned and zero broad-target are the two that matter most for risk.

Gate 5 — The intake gate returns non-conforming requests (behavioural)

Test: During the pilot, deliberately submit an intake request with **no named Application Owner** and a second with **no business justification**.

PASS when both are **returned within the 3-business-day SLA**, not queued for the board, with a clear statement of what is missing. If either reached the ARB agenda, the triage gate in Step 8 is not being enforced — the point of the front door is that it is a filter, not a letterbox.

Gate 6 — The board's decision rights are real (behavioural)

Test: Confirm two things over the pilot's two cycles:

1. Every application that reached `Approved – production` has a matching entry in the

`ARB Decision Record` list with a decision of `Approved` or `Approved with conditions`, recorded within 1 business day.

1. At least one item was **deferred or rejected** (or, if the pilot genuinely had no rejectable

items, that a Security-data item with no Security Reviewer present was correctly deferred).

PASS when approvals and their evidence are traceable one-to-one, and the board demonstrably said "not yet" to at least one item. A board that approves everything is a rubber stamp; the deferral is the proof the gate has teeth.

Gate 7 — Ringed deployment and rollback work (behavioural)

Test: Take one approved pilot application through the full ring progression. Then, on a test application, execute a **rollback** from Ring 1.

PASS when:

• The application was assigned to `PILOT – App Ring 1 – Validation` first, dwelt the minimum time,

and was promoted to Ring 2 and Ring 3 only after meeting the promotion gate in Step 6.

• No approved application in the pilot is assigned directly to *All Users* or *All Devices* (confirm

with Gate 4).

• The rollback was executed by a Deployment Engineer **without requiring any approval**, and

completed within the expected duration recorded in the runbook.

Gate 8 — Change classification is applied consistently (behavioural)

Test: Take three real changes from the pilot — ideally one version update, one new app, and one higher-risk change — and check each against the Step 10 matrix.

PASS when each change's actual approval path matched its tier: the version update went through as a pre-approved standard change (no ARB slot consumed); the new app went through ARB approval; and the higher-risk change carried the additional Security Reviewer sign-off (and CAB notification/approval where applicable). A Tier 1 change that consumed an ARB slot, or a Tier 3 change that skipped Security, is a misclassification to correct in the retrospective.

The overall pass condition

The model is ready to expand (Step 14) when **Gates 1–4 pass automatically** and **Gates 5–8 pass across the two pilot cycles**. Record the results alongside the Step 1 baseline; the contrast between the ungoverned baseline and the passing gates is the evidence you present to the executive sponsor to authorise tenant-wide rollout.

Troubleshooting

This section addresses the ways the operating model stalls in practice — both technical (the scripts and the Intune/Entra objects) and organisational (the process itself). Each entry gives the **symptom**, the **cause**, and the **fix**.

> ***Government tenants:*** *if a script returns no data on a GCC High / DoD tenant, first confirm you passed* `-GraphEnvironment USGov`*; the default* `Global` *endpoint returns nothing against a sovereign tenant. Console URLs are* `endpoint.microsoft.us` *and* `portal.azure.us`*.*

Technical — scripts and objects

Symptom: `Connect-MgGraph` *fails or a script returns no applications*

Cause: the Graph SDK is missing, consent was declined, or (on Government) the wrong environment was targeted.

Fix: confirm the module with `Get-Module Microsoft.Graph -ListAvailable`; install it with `Install-Module Microsoft.Graph -Scope CurrentUser -Force` if absent. Re-run the script and **Accept** the consent prompt. On a Government tenant add `-GraphEnvironment USGov`. If your tenant requires admin consent for the **Microsoft Graph Command Line Tools** enterprise application, have a Global Administrator grant it once (Entra → Enterprise applications → the app → Security → Permissions → Grant admin consent), then re-run.

Symptom: `APPOPS-Validation.ps1` *reports a scope tag, role, or group as MISSING that you believe you created*

Cause: almost always a **name mismatch**. The script matches on the exact display name, including the en dash (–, U+2013) and single spaces — a hyphen-minus (`-`) or a double space fails the match.

Fix: open the object in the console and compare its name character for character with Section 3. The safest correction is to copy the name from the implementation guide and paste it into the object's rename field. Do not "fix" the script's expected names; fix the object so the whole model stays consistent (Testing, scripts, and the register all rely on the exact names).

Symptom: `APPOPS-Remediation.ps1 -Apply` *fails with a 400 or "property cannot be updated"*

Cause: the PATCH body must carry the correct `@odata.type` for the specific app type (`win32LobApp`, `winGetApp`, `iosStoreApp`, etc.). The script reads it from the app and echoes it back, so this normally only occurs if the app object was deleted between the read and the write, or the signed-in account lacks `DeviceManagementApps.ReadWrite.All`.

Fix: confirm the account holds the ReadWrite scope (re-run and check the consent), and re-run the script so it re-reads the current app object. If a specific app persistently fails, stamp its governance metadata manually in **Apps** → **All apps** → **<app>** → **Properties** → **Edit** → **Notes** and apply the scope tag there.

Symptom: `APPOPS-Export-ApprovalEvidence.ps1` *finds no audit events*

Cause: Intune audit-log retention is shorter than the window you requested. Intune retains audit events for a limited period; a change from a year ago will not appear.

Fix: reduce `-DaysBack` to within your tenant's retention, or — for long-term evidence — route Intune audit logs to a Log Analytics workspace (Tenant administration → Diagnostic settings) so the trail outlives the built-in retention. This is worth doing before you rely on the export for a formal audit.

Symptom: the deployment engineer can see apps but cannot edit them

Cause: the app the engineer is trying to edit carries only the **Default** scope tag, which is outside the custom role's assigned scope tags.

Fix: apply `Scope – App Management – Prod` or `Scope – App Management – Pilot` to the app (via `APPOPS-Remediation.ps1` or manually). An app with no management scope tag is, by design, outside the deployment engineers' reach — that is the boundary working, not a bug.

Symptom: creating an Entra group fails with a permissions error

Cause: the tenant restricts security-group creation (Entra → Groups → General → *Users can create security groups* = No) and your account is not in the delegated group or a directory role.

Fix: have a Global Administrator or Groups Administrator either assign you **Groups Administrator** or add you to the group nominated in the group-creation setting (Requirement R4).

Organisational — the process

Symptom: the board approves everything; nothing is ever deferred or rejected

Cause: the evidence pack is not being demanded before items are tabled, so the board is presented only with items that already look complete — and it has no basis to say no. Or the sponsor has not made it safe to reject.

Fix: enforce the evidence-pack requirement (Step 9, item 6) — the chair refuses to table an item missing any required evidence. Confirm the sponsor (R1) will back a rejection. A board that never defers is not disciplined; it is a rubber stamp, and Testing Gate 6 will not pass.

Symptom: urgent applications keep bypassing the intake form

Cause: urgency is being treated as an exemption rather than a change tier.

Fix: reframe urgency inside the model — an urgent application is a Tier 3/Tier 4 change with compressed dwell times and immediate Security involvement, handled *fast* but not *outside* the process. Have the executive sponsor restate that intake is the only route. Every tolerated bypass teaches the next requester that the bypass exists.

Symptom: the board is drowning in trivial version-update approvals

Cause: the Tier 1 pre-approval (standard change) rule was not implemented, or is defined too narrowly.

Fix: implement Step 9 item 7 — a version update to an approved app with no change to data handling, egress, or licence is pre-approved and consumes no ARB slot. Widen the standard-change definition as confidence grows. This one rule typically removes 60–80% of the board's volume and is the difference between a viable board and an abandoned one.

Symptom: applications keep becoming orphaned despite the register

Cause: the leaver process does not reassign a departing owner's applications.

Fix: add "reassign owned applications in the register" to the IT leaver checklist, and have the register custodian run the overdue/unowned sweep monthly (`APPOPS-Discovery.ps1`, the *No governed owner* count). An owner who has left is the single most common route to orphaned applications; catch it at the leaver event, not at the next audit.

Symptom: the register and Intune disagree about what exists

Cause: applications are installed on endpoints outside Intune (user-installed, legacy image, departed process) and were never reconciled.

Fix: run the quarterly reconciliation (Requirement R8) — compare Intune **Discovered apps** (and Defender for Endpoint software inventory where available) against the register, and either bring each shadow application into governance or plan its removal. Track the shadow count on the scorecard so it trends down rather than being rediscovered at each audit.

Symptom: nobody will hold the Security Reviewer role

Cause: the organisation has no security function, or Security declines the workload.

Fix: escalate to the sponsor (R1). Where a security function genuinely does not exist, the model still runs, but record the **compensating control** explicitly in the standards document — for example, a required external annual assessment for any `Confidential`/`Restricted` application — rather than pretending the gate exists. A gate with no owner is not a gate.

Symptom: the model was built, then quietly stopped being used after a few months

Cause: the scorecard is not reported anywhere with authority, so nobody notices the decay.

Fix: put the monthly `APPOPS-Report.ps1` scorecard on the standing agenda of a governance forum that can act on it, and show the **trend**, not just the snapshot. A falling ownership-coverage line in front of the right people prompts correction; the same number in a folder does not. This is the maintenance practice that most determines whether the model survives its first year.

Best Practices

These are the disciplines that keep the operating model alive after the initial build. Governance programmes rarely fail at launch — they fail six months later when the board stops meeting and the register stops being maintained. Each practice below counters a specific way that happens.

Secure the sponsor before the process, not after

The most common cause of failure is standing up the board before securing R1's executive sponsor. The first time the board rejects a director's preferred application, an unsponsored board folds and becomes advisory. Get the sponsorship email in hand, and make sure the sponsor understands that their job is precisely to absorb that one uncomfortable escalation. One backed rejection in the first quarter is worth more to the model's credibility than a hundred approvals.

Make the front door the only door

An intake form that people can route around is theatre. The moment a "just this once, it's urgent" application bypasses intake, every subsequent requester learns that urgency is the bypass. Handle

urgency **inside** the model — an urgent application is a Tier 3 or Tier 4 change with compressed dwell times and immediate Security involvement, not an exemption from governance. Have the executive sponsor, not IT, announce that intake is mandatory; it lands differently.

Pre-approve the routine so the board can judge the exceptional

The Tier 1 standard-change rule (Step 9, item 7) is not a convenience — it is what keeps the board viable. If every Chrome version bump needs an ARB slot, the board drowns, meetings overrun, and people start bypassing it out of sheer friction. Pre-approving low-risk version updates typically removes 60–80% of the volume and lets the board spend its attention on the decisions that actually carry risk. Review the standard-change definition quarterly; it should be as wide as is safe.

Name individuals, never teams

A RACI that says "Endpoint Team" for Technical Owner has no owner. When something breaks, "the team" is nobody. Every application in the register carries a **named person** in each accountability. When that person leaves, the register custodian's leaver process reassigns their applications explicitly — this is the single most valuable maintenance task the custodian performs, because unreassigned leavers are exactly how applications become orphaned.

Treat the Security Reviewer as non-negotiable for classified data

The role most often quietly dropped is Security Reviewer, because IT can approve without them and usually wants to move faster. Resist this. The whole point of the model is that an application handling `Confidential` or `Restricted` data does not reach production without a security determination. Build the quorum rule (no Security Reviewer present → such items are deferred) into the board's standing behaviour so it is automatic, not a judgement call in the moment.

Ring everything; assign nothing to All Users

Direct assignment to *All Users* or *All Devices* is the single behaviour that makes a bad change catastrophic. Ringed deployment turns a broken package into a Ring 1 incident affecting a handful of validators instead of a fleet-wide outage. Enforce it structurally: the deployment engineers' muscle memory should be "which ring", never "everyone". The validation script flags broad targeting specifically because it is the highest-leverage risk to eliminate.

Make rollback approval-free and cheap

The most expensive change-management mistake is requiring approval to roll back. A rollback is always a safe action — it returns to a known-good state. Requiring a meeting to authorise it converts a five-minute recovery into a two-hour outage while people are found and convened. State the principle loudly and repeatedly: **any deployment engineer may roll back any change at any time.** The dwell times exist precisely so that a rollback is still cheap when you need it.

Reconcile against reality, not against your own deployment records

Intune tells you what you *deployed*. It does not tell you what is *installed*. Users install software, legacy images carry software, and departed processes left software behind. The quarterly reconciliation of Intune

Discovered apps (and Defender for Endpoint software inventory where available) against the register is how you find the ungoverned estate. The "shadow application" count is uncomfortable precisely because it is honest; track it as a first-class risk metric and drive it down.

Publish the scorecard where decisions are made

Metrics reported nowhere change nothing. The governance scorecard (`APPOPS-Report.ps1`) belongs on the standing agenda of a forum with authority — an IT operations review or a service-management board — so that a declining ownership-coverage trend prompts action rather than sitting in a folder. Show the trend, not just the snapshot; the month-over-month direction is what tells you whether the model is winning or slowly decaying.

Let the standards document be a living test, not a monument

The standards document (Step 7) is only useful while it reflects reality. After each pilot retrospective, and at least annually, revise it: tighten a criterion that let a bad app through, loosen one that generated rework for no risk reduction, and widen the standard-change definition as your confidence grows. Version it, and record who approved each version. A standards document that has not changed in two years is either perfect or ignored — and it is not perfect.

Retire deliberately; a smaller estate is the goal

The model's success is not a bigger, tidier register — it is a **smaller** estate. Every application carries licence cost, security surface, support load, and review effort. Applications no owner will claim are applications the organisation does not need; move them to `Deprecated`, communicate a 60-day notice, and retire them. The delta between your Step 1 baseline and your steady-state scorecard — fewer apps, all owned, all reviewed, all ringed — is the entire business case, so measure it and report it at the annual review.

Deliverables Summary

This solution delivers a complete application management **operating model** — the governance layer above packaging and deployment — for a Microsoft Intune / Entra ID estate, with the tooling to stand it up and keep it measured.

Documents

Document	What it gives you
Solution Overview (`01-solution-overview.md`)	The seven components of the operating model, how each governance decision maps to a concrete Intune/Entra object, the interface to WDAC/MDE/App Protection, and the honest scope boundaries.
Global Requirements (`global-requirements-intune-solutions.md`)	Category-level prerequisites (tenant, Intune, enrolment, Windows baseline, network).

Document	What it gives you
Requirements (`02-requirements.md`)	Ten solution-specific prerequisites — executive sponsor, admin and least-privilege roles, group-creation rights, Graph SDK, a system of record, the CAB position, discovery sources, named role holders, and licence data — each with Description / Why / Verify / Implement.
Implementation (`03-implementation.md`)	Fourteen ordered steps: baseline the estate, build the register, define the RACI, create the four role groups, the two scope tags and the least-privilege custom role, the three deployment rings, the standards document, the intake workflow, the review board, the change classification matrix and CAB interface, the support model and runbook standard, stamp governance onto the existing estate, pilot for two board cycles, and expand to steady state.
Testing (`05-testing.md`)	Eight pass/fail gates — four structural (automated) and four behavioural — that prove the model holds under real traffic before you expand.
Troubleshooting (`06-troubleshooting.md`)	Fixes for the ways the model stalls in practice — script access, missing objects, quorum failures, bypass attempts, orphan reconciliation.
Best Practices (`07-best-practices.md`)	The ten disciplines that keep the model alive past month six, each countering a specific failure mode.
Executive Summary (`executive-summary.md`)	The business case in one page, for securing the sponsor.

PowerShell scripts

All scripts use the Microsoft Graph PowerShell SDK and accept `-GraphEnvironment USGov` for Government tenants. Four of the five are strictly **read-only**; only Remediation writes, and it defaults to `-WhatIf`.

Script	Purpose	Access
APPOPS-Discovery.ps1	Inventory the estate and surface the governance gaps (orphaned, broad-target, unowned, unscoped). Seeds the register and the baseline.	Read-only

Script	Purpose	Access
APPOPS-Validation.ps1	Test the estate against the model's structural gates — scope tags, custom role, the seven groups, and per-app naming/owner/targeting conformance.	Read-only
APPOPS-Report.ps1	Produce the monthly Application Governance Scorecard — ownership, scope-tag, and naming coverage plus broad-target and orphaned risk counts, appendable for trending.	Read-only
APPOPS-Export-ApprovalEvidence.ps1	Export the change-approval evidence trail (owner block, scope tags, ring assignments, audit events) as the ARB/audit evidence pack.	Read-only
APPOPS-Remediation.ps1	Stamp owner + ARB reference + scope tag onto existing applications from a CSV. Changes no deployment behaviour.	`-WhatIf` default

Artefacts the model produces (and you keep)

The Application Register, the ARB Decision Record, the RACI matrix, the Application Standards Document, the Change Classification Matrix, the Support Matrix, the per-application Runbook, and the Governance Scorecard — all defined in the implementation guide with exact schemas and named objects.

How the pieces fit

1. Confirm **Requirements** — above all, secure the executive sponsor.
2. Run **Discovery** to baseline the estate and quantify the gaps.
3. Build the **objects** (register, RACI, groups, scope tags, role, rings, standards, intake, board, change matrix, support model) per the implementation guide.
1. Stamp governance onto the existing estate with **Remediation** (gated).
2. **Pilot** on one business unit for two board cycles; prove it with the **Testing** gates via **Validation**.
1. **Expand** tenant-wide; run **Report** monthly and **Export-ApprovalEvidence** for audit, and drive the estate down through deliberate retirement.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

APPOPS-Discovery.ps1

Purpose: Read-only. Builds the "before" picture for the application management operating model. For every application in Intune it reports the assignment count, whether it targets All Users / All Devices (a governance red flag), its scope tags, and whether a governance owner block has been stamped into its Notes field. This is the baseline the Application Register is seeded from and the scorecard is measured against. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPOPS-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\APPOPS-Discovery.ps1 -CsvPath .\AppEstate-Baseline.csv Government tenant: add -GraphEnvironment USGov Required Graph scopes: DeviceManagementApps.Read.All, DeviceManagementRBAC.Read.All, Group.Read.All EXPECTED RESULTS A table (and optional CSV) of every application with: AssignmentCount, TargetsAllUsers, TargetsAllDevices, ScopeTags, GovernedOwner. Pass = every production application has an assignment, no All Users/All Devices targeting, a non-default scope tag, and a governed owner. Fail = any blank owner, zero assignment, or All Users/All Devices targeting — these are your ungoverned / orphaned applications to remediate. .NOTES Intune Solutions — read-only. Uses the Microsoft Graph PowerShell SDK. >

How to run: powershell -ExecutionPolicy Bypass -File .\APPOPS-Discovery.ps1 powershell -ExecutionPolicy Bypass -File .\APPOPS-Discovery.ps1 -CsvPath .\AppEstate-Baseline.csv Government tenant: add -GraphEnvironment USGov Required Graph scopes: DeviceManagementApps.Read.All, DeviceManagementRBAC.Read.All, Group.Read.All EXPECTED RESULTS A table (and optional CSV) of every application with: AssignmentCount, TargetsAllUsers, TargetsAllDevices, ScopeTags, GovernedOwner. Pass = every production application has an assignment, no All Users/All Devices targeting, a non-default scope tag, and a governed owner. Fail = any blank owner, zero assignment, or All Users/All Devices targeting — these are your ungoverned / orphaned applications to remediate. .NOTES Intune Solutions — read-only. Uses the Microsoft Graph PowerShell SDK. >

APPOPS-Export-ApprovalEvidence.ps1

Purpose: Read-only. For a named application (or all applications), export the audit evidence an Application Review Board or an auditor needs to prove a change was authorised: the current governance Notes block, the applied scope tags, the assignment groups (i.e. which rings it reached), and the Intune audit events that recorded who created, assigned, or modified the app and when. This is the "evidence pack" the ARB attaches to each Decision Record. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPOPS-Export-ApprovalEvidence.ps1 -AppNameLike "Acrobat" powershell -ExecutionPolicy Bypass -File .\APPOPS-Export-ApprovalEvidence.ps1 -AppNameLike "Acrobat" -OutFolder .\Evidence -DaysBack 180 Government tenant: add -GraphEnvironment USGov Required Graph scopes: DeviceManagementApps.Read.All, DeviceManagementConfiguration.Read.All EXPECTED RESULTS A per-application evidence file (CSV of audit events + a text summary of owner/scope/ assignments) written to the output folder. Pass = a complete, dated trail of who changed the app. Fail = no audit events found (audit retention may be shorter than the requested window — verify Tenant administration > Audit logs retention). .NOTES Intune Solutions — read-only. Uses the Microsoft Graph PowerShell SDK. >

How to run: powershell -ExecutionPolicy Bypass -File .\APPOPS-Export-ApprovalEvidence.ps1 -AppNameLike "Acrobat" powershell -ExecutionPolicy Bypass -File .\APPOPS-Export-ApprovalEvidence.ps1 -AppNameLike "Acrobat" -OutFolder .\Evidence -DaysBack 180

Government tenant: add -GraphEnvironment USGov Required Graph scopes: DeviceManagementApps.Read.All, DeviceManagementConfiguration.Read.All EXPECTED RESULTS A per-application evidence file (CSV of audit events + a text summary of owner/scope/ assignments) written to the output folder. Pass = a complete, dated trail of who changed the app. Fail = no audit events found (audit retention may be shorter than the requested window — verify Tenant administration > Audit logs retention). .NOTES Intune Solutions — read-only. Uses the Microsoft Graph PowerShell SDK. >

APPOPS-Remediation.ps1

Purpose: The one WRITE script in this package. It reads a CSV of applications and writes a structured governance block into each application's Notes field (Owner, ARB reference, governed date) and applies the named scope tag. It changes NOTHING about the application's package, assignments, or deployment behaviour — it only records who owns the app and that it was governed. Defaults to -WhatIf: it previews and writes nothing until you re-run with -Apply. HOW TO RUN Preview (writes nothing): powershell -ExecutionPolicy Bypass -File .\APPOPS-Remediation.ps1 -InputCsv .\AppEstate-Governance.csv Apply: powershell -ExecutionPolicy Bypass -File .\APPOPS-Remediation.ps1 -InputCsv .\AppEstate-Governance.csv -Apply Government tenant: add -GraphEnvironment USGov Required Graph scopes: DeviceManagementApps.ReadWrite.All, DeviceManagementRBAC.Read.All

How to run: Preview (writes nothing): powershell -ExecutionPolicy Bypass -File .\APPOPS-Remediation.ps1 -InputCsv .\AppEstate-Governance.csv Apply: powershell -ExecutionPolicy Bypass -File .\APPOPS-Remediation.ps1 -InputCsv .\AppEstate-Governance.csv -Apply Government tenant: add -GraphEnvironment USGov Required Graph scopes: DeviceManagementApps.ReadWrite.All, DeviceManagementRBAC.Read.All

APPOPS-Report.ps1

Purpose: Read-only. Computes the operating model's steady-state metrics across the whole estate: ownership coverage, scope-tag coverage, ring-assignment coverage, broad-target risk, naming conformance, and the shadow/orphaned counts. These are the numbers you report monthly to the governance forum to prove the model is working — a rising coverage trend and a falling risk trend. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPOPS-Report.ps1 powershell -ExecutionPolicy Bypass -File .\APPOPS-Report.ps1 -CsvPath .\Scorecard.csv Government tenant: add -GraphEnvironment USGov Required Graph scopes: DeviceManagementApps.Read.All, DeviceManagementRBAC.Read.All, Group.Read.All EXPECTED RESULTS A one-page scorecard of coverage percentages and risk counts, plus an optional CSV row you can append month over month to chart the trend. Pass = coverage climbing toward 100% and risk counts trending to zero over successive runs. .NOTES Intune Solutions — read-only. Uses the Microsoft Graph PowerShell SDK. >

How to run: powershell -ExecutionPolicy Bypass -File .\APPOPS-Report.ps1 powershell -ExecutionPolicy Bypass -File .\APPOPS-Report.ps1 -CsvPath .\Scorecard.csv Government tenant: add -GraphEnvironment USGov Required Graph scopes: DeviceManagementApps.Read.All, DeviceManagementRBAC.Read.All, Group.Read.All EXPECTED RESULTS A one-page scorecard of coverage percentages and risk counts, plus an optional CSV row you can append month over month to chart the trend. Pass = coverage climbing toward 100% and risk counts trending to zero over successive runs. .NOTES Intune Solutions — read-only. Uses the Microsoft Graph PowerShell SDK. >

APPOPS-Validation.ps1

Purpose: Read-only. Checks whether the governance controls this solution defines actually exist and are applied. It validates: the two scope tags exist (Scope - App Management - Prod / - Pilot); the custom role exists (Role - App Deployment Engineer); the four APPGOV role groups and three ring groups exist; and every application conforms to the naming standard, has a governed owner, and is not assigned to All Users / All Devices. Produces a PASS/FAIL line per gate. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\APPOPS-Validation.ps1 powershell -ExecutionPolicy Bypass -File .\APPOPS-Validation.ps1 -CsvPath .\Validation.csv Government tenant: add -GraphEnvironment USGov Required Graph scopes: DeviceManagementApps.Read.All, DeviceManagementRBAC.Read.All, Group.Read.All EXPECTED RESULTS A gate-by-gate PASS/FAIL report and an overall verdict. Pass = every structural gate present and every application conformant. Fail = any missing object or any non-conforming application, itemised so you can remediate the exact gap. .NOTES Intune Solutions — read-only. Uses the Microsoft Graph PowerShell SDK. >

How to run: powershell -ExecutionPolicy Bypass -File .\APPOPS-Validation.ps1 powershell -ExecutionPolicy Bypass -File .\APPOPS-Validation.ps1 -CsvPath .\Validation.csv Government tenant: add -GraphEnvironment USGov Required Graph scopes: DeviceManagementApps.Read.All, DeviceManagementRBAC.Read.All, Group.Read.All EXPECTED RESULTS A gate-by-gate PASS/FAIL report and an overall verdict. Pass = every structural gate present and every application conformant. Fail = any missing object or any non-conforming application, itemised so you can remediate the exact gap. .NOTES Intune Solutions — read-only. Uses the Microsoft Graph PowerShell SDK. >